

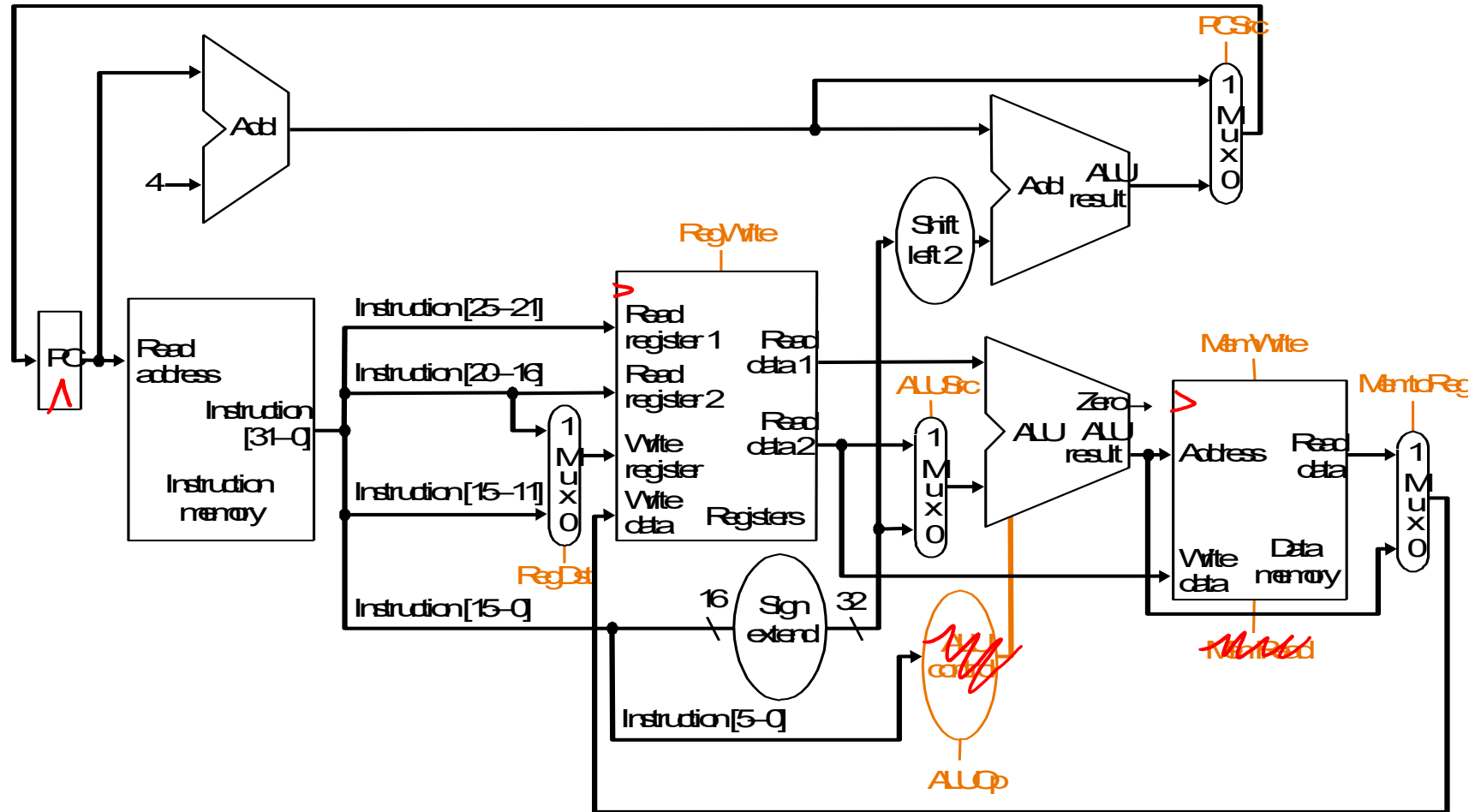
- Midterm

Monday 5/11

- Lab 3

progress check  
today

# The single-cycle processor

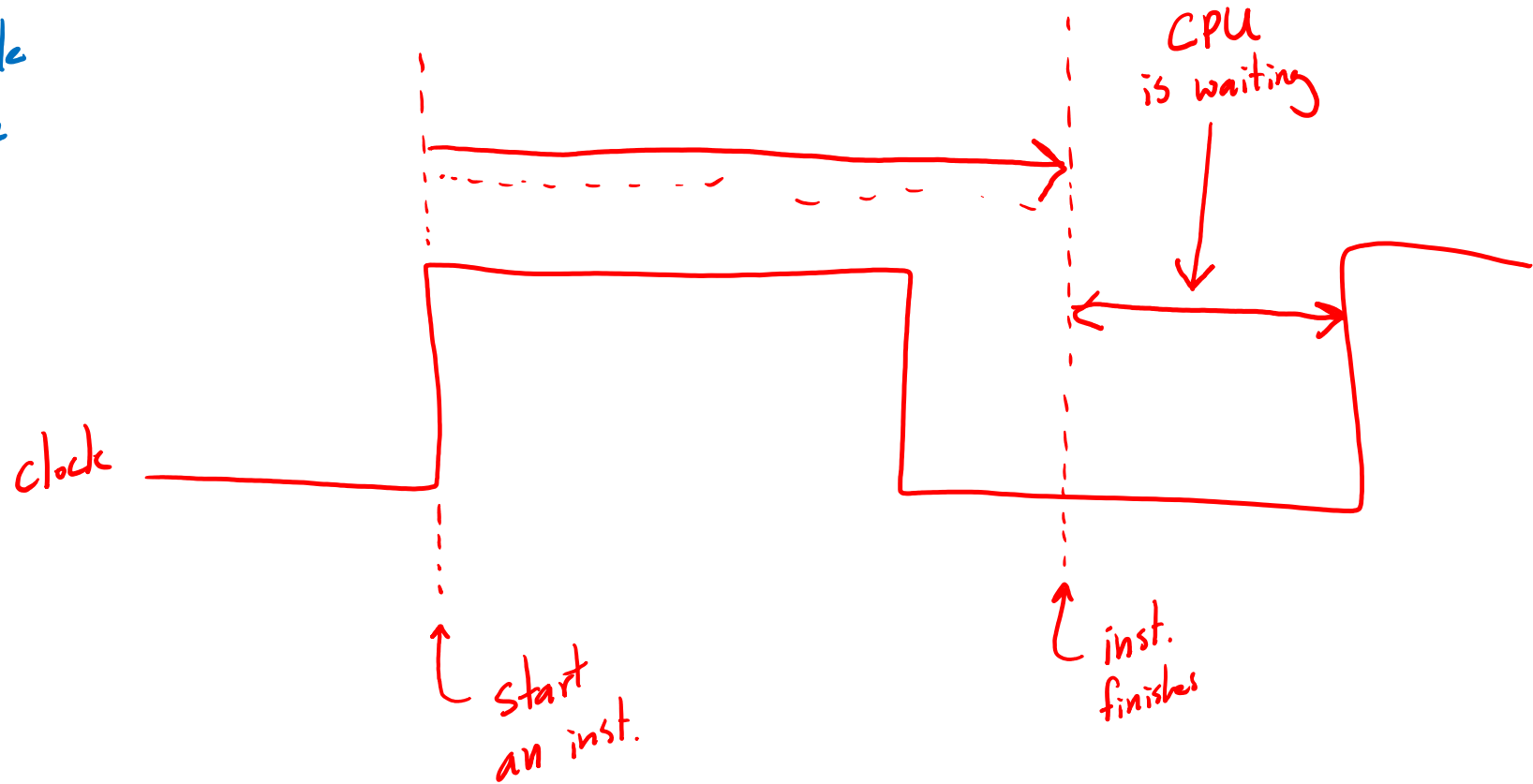


# Pipelining

4.5 - 4.6

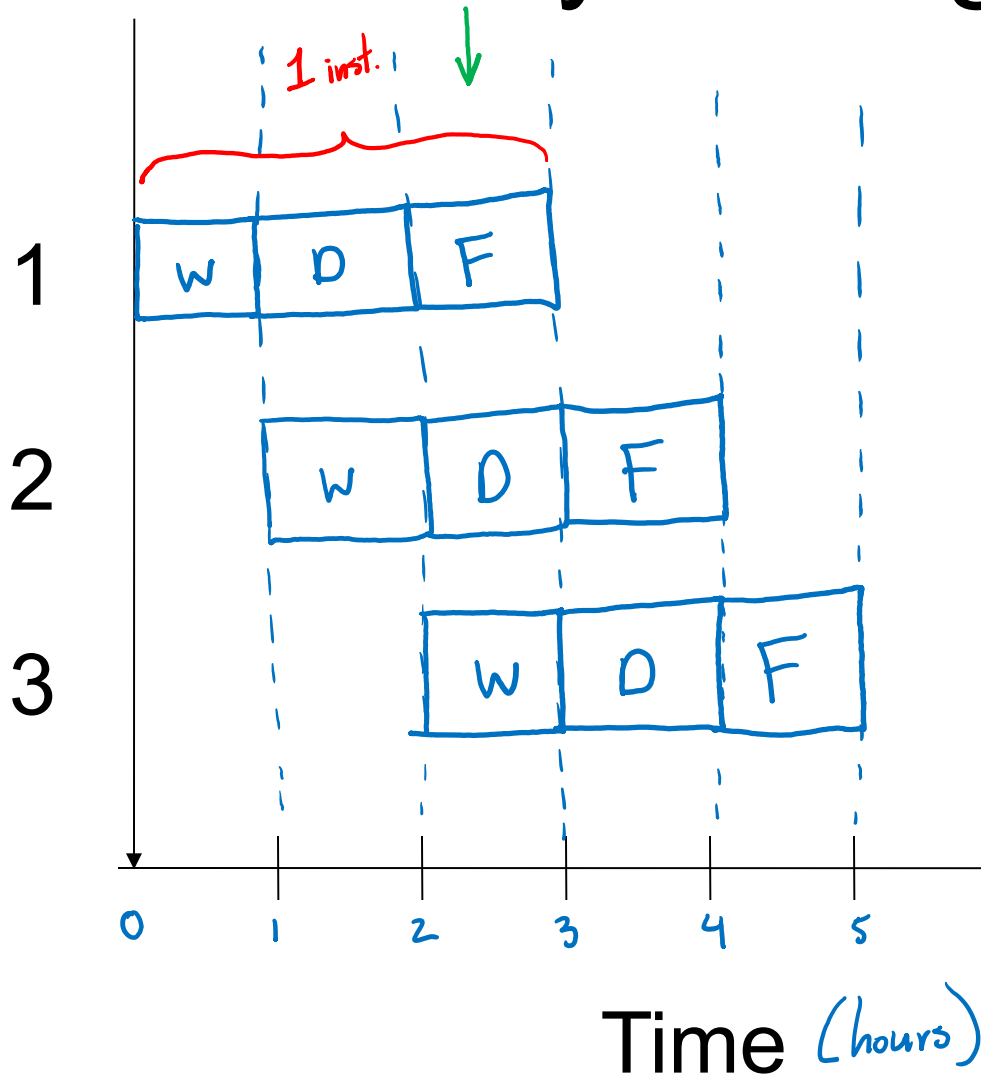
# Datapath Progression

- single cycle  
CPUs are  
slow



# Laundry Analogy

Loads  
of laundry



1 washer  
1 dryer  
1 folder } each task takes 1 hour

- 3 loads in 5 hours
- each load still takes 3 hours
- in the steady state, a load finishes every hour

# Pipelined Processing

- Start a new inst. every clock
- Time to complete an inst. does not change
- Multiple inst. "in flight" at the same time
- An inst. completes every clock cycle

# Why pipelining is faster

Program  
has 100 inst.

- Single-cycle:

(100 MHz)



10 ns

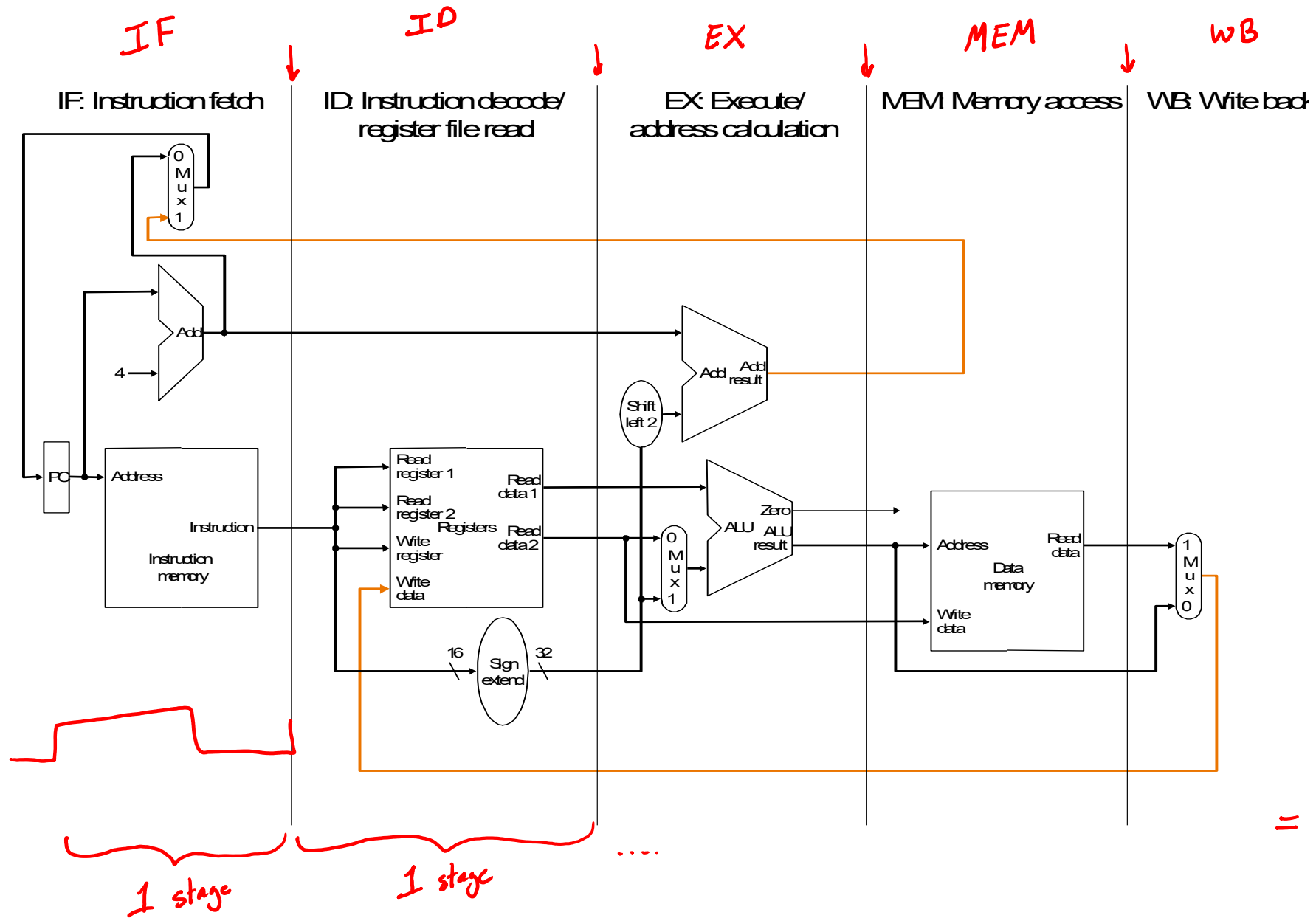
$$10 \text{ ns} \cdot 100 \text{ inst} \cdot 1 \text{ CPI} = 1000 \text{ ns}$$

↑  
clock  
cycle

- Pipelined:

$$2 \text{ ns} \cdot 100 \text{ inst} \cdot 1 \text{ CPI} + \underbrace{4 \text{ clock cycles} \cdot 2 \text{ ns}}_{\text{fill time}} = 208 \text{ ns}$$

↑  
clock  
cycle

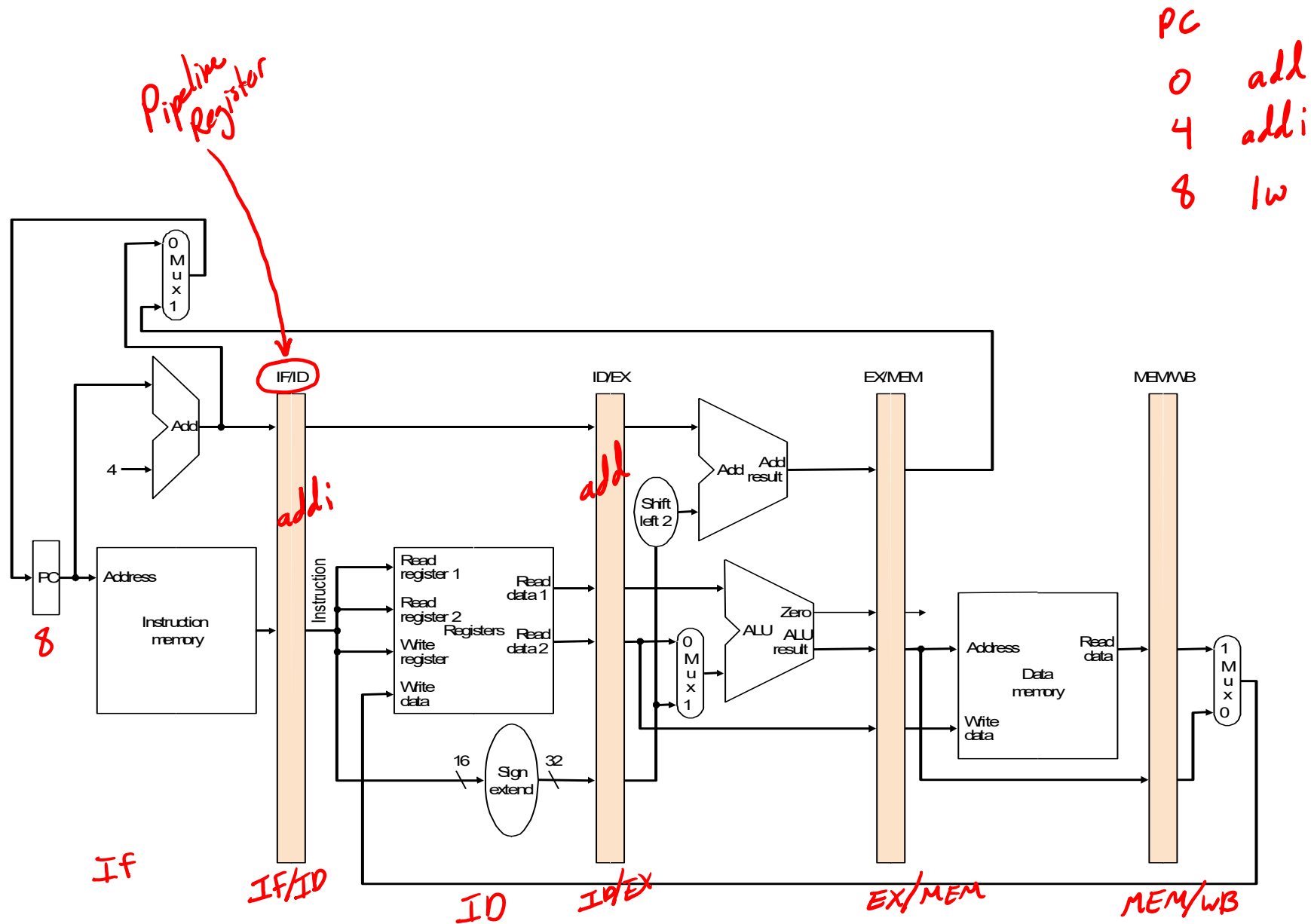




# Inter-stage registers

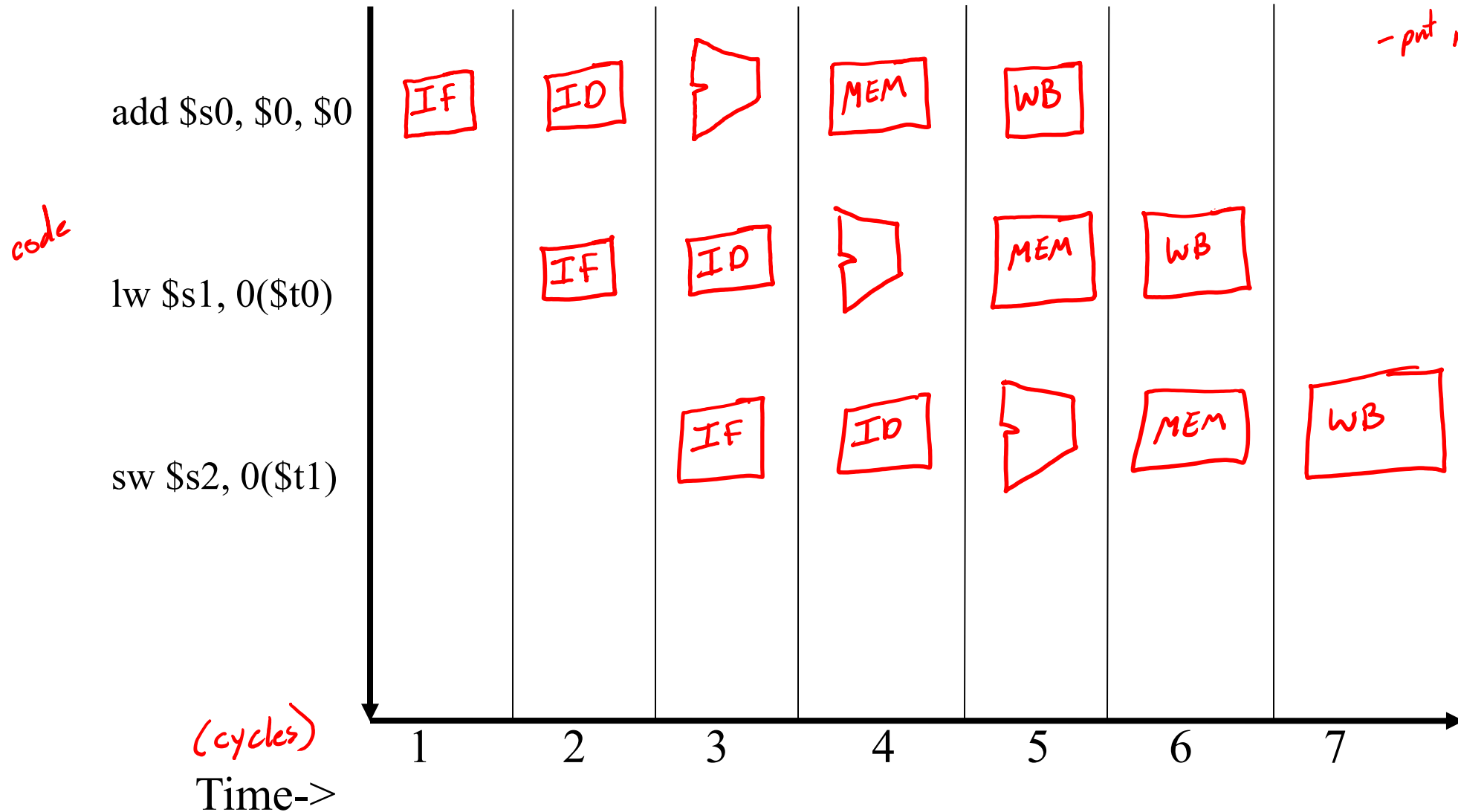
( Pipeline Register)

- hold all the information about each inst.
- named after the stages they are between



# Pipelined operation

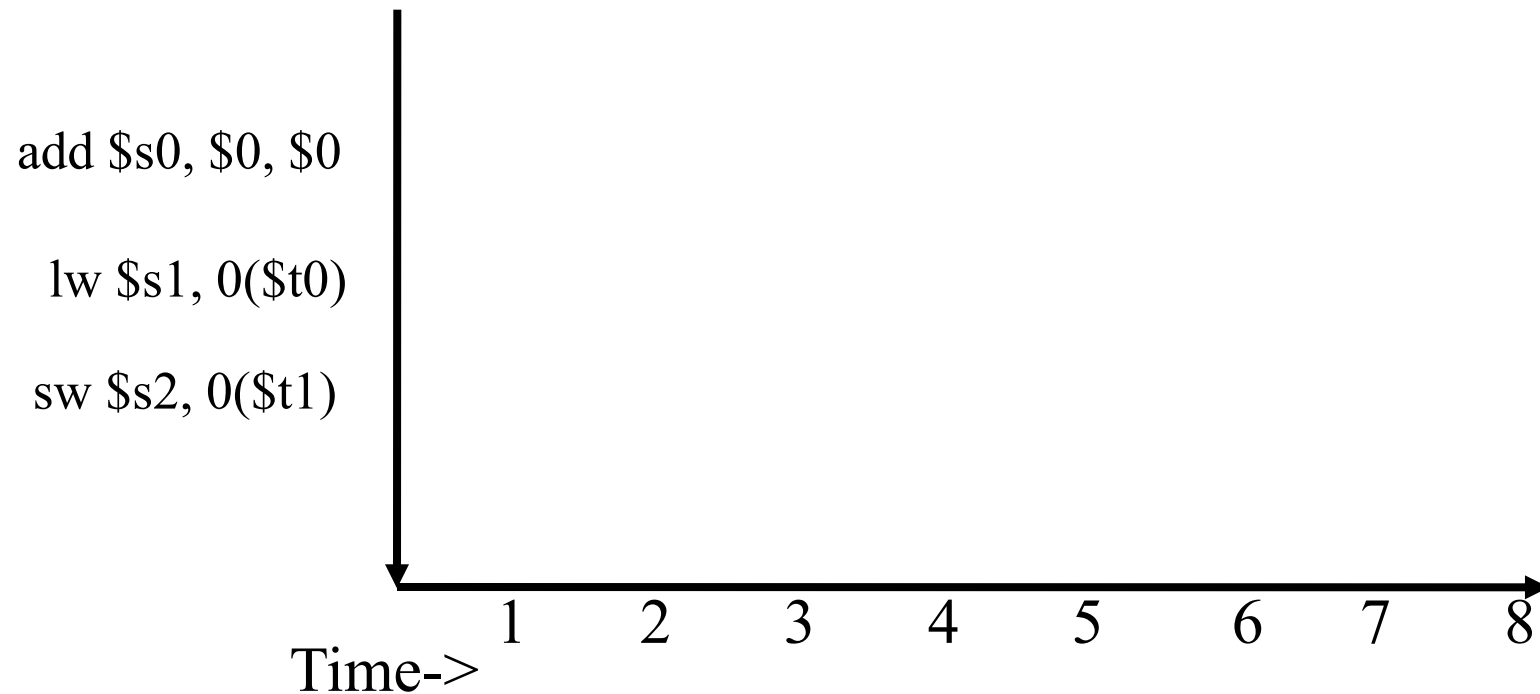
- do not use package
- turn in Makefile
- put name in files



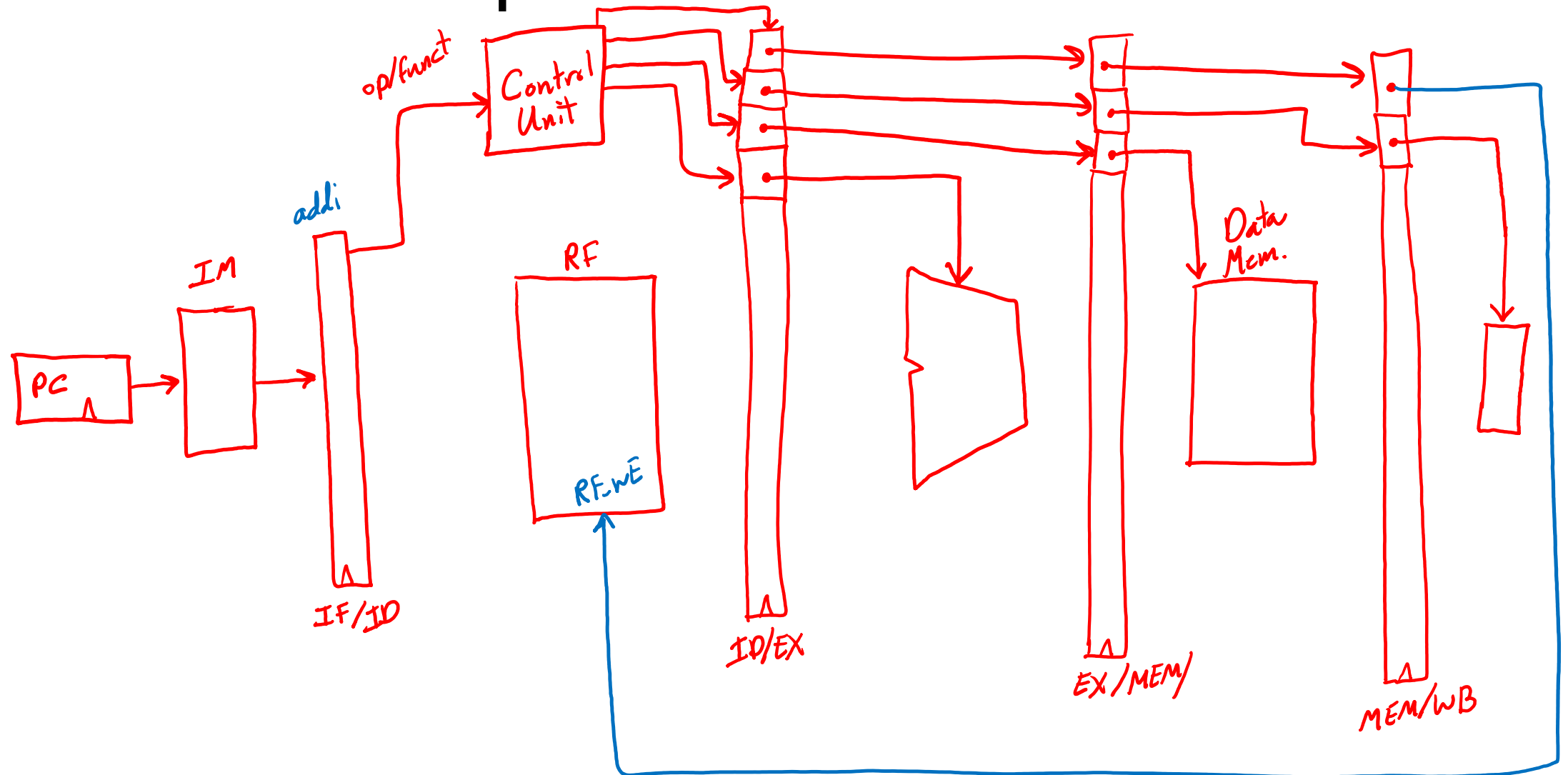
In what cycle was \$s1 written? 6

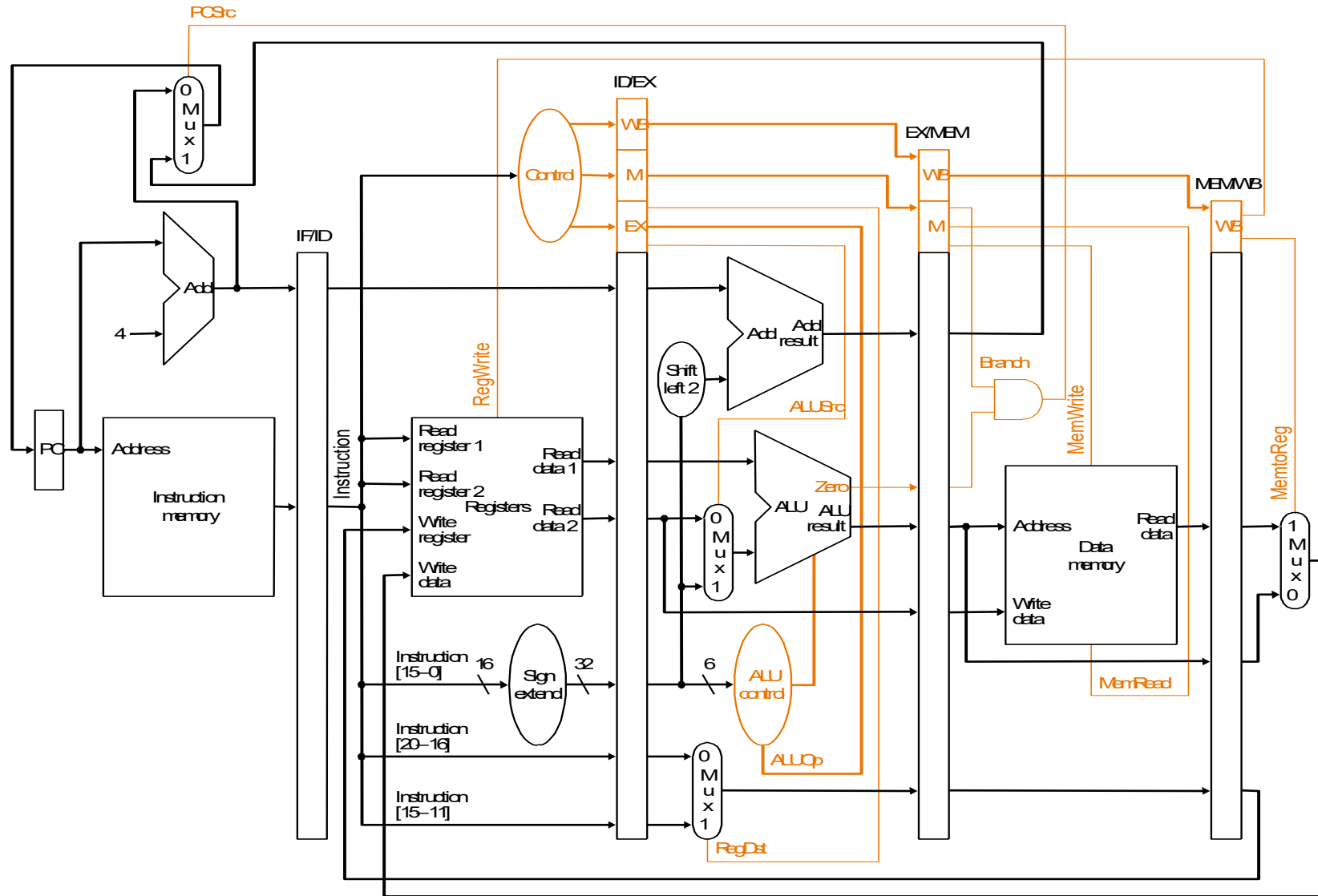
In what cycle was \$t1 read? 4

In what cycle was the Add executed? 3



# Pipelined control





add \$s0, ..., ~  
 add ..., \$s0, ...

# Problems with pipelining

## Hazards

1. Structural Hazards — multiple inst. want to use the same structure at the same time
2. Data Hazards — an inst. depends on another inst.  
↑  
it is still in the pipeline
3. Control Hazards — branches and jumps